

OCP KOREA
TECH DAY 2026

GitAIOps

AI 에이전트에게 지워지지 않는 기억을 주는 3층 구조

조훈 **Hoon Jo** CNCF & AAIF Ambassador | AI & Cloud Native Engineer,
MegazoneSoft

OCP Korea Tech Day 2026 | 8월 21일 | 코엑스 그랜드볼룸



LIVE DEMO 저장소에 질문합니다



gitaioops/
notiflex-platform

여기 저장소가 하나 있습니다

B2B 알림 SaaS 플랫폼 GKE 위에 구축된 완성본

책의 전 과정을 구축 완료 결정 기록(ADR) 16건이 쌓여 있음

위키를 쓴 사람은 없음 그런데 무엇이든 물어보면 답합니다



gitaioops/
notiflex-platform

여러분이라면 무엇을 물어보시겠습니까?

지금 바로 실행합니다.

질문이 없다면, 이 질문부터 시작합니다.

준비된 질문: Why was Alertmanager chosen over Grafana Alerting?

Git "AI" Ops

GitOps 선언한 상태를 Git에 두면, 도구가 그 상태로 맞춰줍니다

GitAIOps 그 한가운데에 AI를 넣고, 지워지지 않는 기억을 줍니다





Who am I ?



<https://github.com/SysNet4Admin>



<https://www.linkedin.com/in/hoonjo/>

방금 그 답은 어디서 왔을까요?

모델이 똑똑해서가 아닙니다. 원래 AI 에이전트는 세션이 끝나면 전부 잊습니다.

잊는다 어제의 결정, 값, 교훈이 다음 세션에는 남아 있지 않다

추측한다 그래서 빈칸을 추측으로 메운다. 끝난 결정을 다시 정하고, 명령을 지어낸다

예측이 안 된다 같은 요청에 다른 결과. production은 그 위에서 돌아갈 수 없다

그런데 방금 그 저장소는 답했습니다. 기억이 저장소에 있기 때문입니다.



CLAUDE.md 항상 지켜야 할 규칙

CLAUDE.md

항상 지켜야 할 규칙

매 세션 로드

에이전트가 대화를 시작할 때마다 자동으로 읽는 규칙

무엇이 담기나

저장소 구조, 작업 원칙, 하지 말아야 할 것

CLAUDE.md 실제 내용

행동 규칙

- 항상 현재 상태를 확인한 후 작업을 진행한다
- `kubectl` 명령은 반드시 `--context` 를 지정한다

파일 이름은 도구를 따라갈 뿐입니다. 구조는 **Git**에 있지, 도구에 있지 않습니다.



claude-context/ 현재 아키텍처의 전체 그림

claude-context

현재 아키텍처의 전체 그림

필요할 때 참조

지금 무엇이 어디에 어떻게 돌고 있는지, 스냅샷 한 벌

교과서가 아니라 요약노트

긴 계획서 대신, 시험 범위만 추린 상태 대시보드

claude-context/architecture.md 실제 내용

클러스터 notiflex-cluster (asia-northeast3-a)
노드풀 default-pool / api-pool / worker-pool / ops-pool
GKE 기능 Gateway API, Workload Identity, Secret Manager CSI

사람과 AI가 같은 문서를 보고, 같은 내용을 공유합니다.



docs/ (ADR) 결정의 누적 기록

docs

결정의 누적 기록

16건의 결정

무엇을 골랐고, 무엇을 버렸고, 왜 그랬는지까지

사람과 AI가 함께 검토

결정이 생길 때마다 순서대로 쌓인다

`docs/architecture-decisions.md` 실제 내용

ADR-005: 알림 - PrometheusRule (ch4.4)

결정: PrometheusRule CRD 채택 (vs Grafana Alert)

이유: PromQL 정밀 조건 / git 버전 관리 / Alertmanager 라우팅 결합

오프닝에서 보신 그 답의 출처가 바로 여기입니다.



세 층의 구분 기준: 역할과 로드 시점

층	역할	로드 시점
CLAUDE.md	항상 지켜야 할 규칙	매 세션 로드
claude-context/	현재 아키텍처의 전체 그림	필요할 때 참조
docs/ (ADR)	결정의 누적 기록	사람과 AI가 함께 검토

실행 순서로 나눈 것이 아닙니다. 역할과 로드 시점으로 나눈 것입니다.



기록은 이벤트가 아니라 루틴입니다

1

작업한다

2

결정이 생긴다

3

ADR로 남긴다

4

context를 갱신한다

그리고 다음 세션이 이 기록을 그대로 이어받습니다.

JOURNEY.md 무엇이 언제 어떻게 쌓였는지, 축적 자체의 기록

한 번의 정리가 아니라, 매 작업 끝의 습관입니다.



이번엔 더 어려운 걸 시켜보죠





gitaioops/
notiflex-platform

새로 합류한 엔지니어가 첫 주에 할 일을 정리해줘

없던 문서를, 저장소에 쌓인 기록만으로 그 자리에서 만듭니다



gitaioops/
notiflex-platform

Ask anything

남은 질문이 있다면 지금이 기회입니다

예비 질문: Why was Gateway API chosen over Ingress?

방금 보신 것

여러분의 질문에 그 자리에서 답했다 준비된 각본 없이, 기록된 결정에서

없던 문서가 그 자리에서 나왔다 ADR과 가드레일, 트러블슈팅 이력까지 읽어서

저장소가 단일 진실 공급원이 됐다 규칙, 현재 상태, 결정. 세 층이 전부 저장소 안에

위키를 쓴 사람은 아무도 없습니다.



여러분의 것으로: 작게 시작

- 1. 지금 시작한다** 규칙과 현재 상태를 기록하는 것부터. 도구가 읽는 파일이면 무엇이든
- 2. 작업이 커지면 계획부터** 정해진 것을 옮기는 작업이라면 계획을 문서로, 값까지 고정해서
- 3. 기록을 살아 있게** 쓰는 사람 모두가 갱신한다. AI가 읽기 좋은 형태는 AI의 도움으로

이 구조는 Git에 있는 것이지, 특정 도구에 있는 것이 아닙니다.
프로젝트가 끝나도 기억은 계속 쓰입니다.



정리하겠습니다



GitAIOps, 세 문장으로

Git

Git is the memory

모든 규칙, 상태, 결정이 저장소에 남는다

AI

AI is the operator

3층 지식 구조가 읽고 쓰는 순서를 만든다

Ops

Ops is predictable

단일 진실 공급원에서 나오는 운영

감사합니다!



GitAIOps on GitHub
github.com/gitaiops



실습은 책 저장소에서
github.com/sysnet4admin/_Book_GitAIOps



조훈 Hoon Jo
CNCF & AAIF Ambassador

